

Exploratory Thoughts: Unified Recommender System Architecture

Transformer Sequence Modeling + Feature Interaction

algo.qq.com

David Xia

April 2026

Draft v0.02

Prepared in the context of the QQ Algorithm Challenge — algo.qq.com

Abstract

These are first-thoughts towards a unified recommender system architecture that merges transformer-based sequence modelling with structured feature interaction, targeting the QQ Algorithm Challenge. This version explores a TensorFlow/Keras implementation. The intent is exploratory: to sketch the mathematical and implementation foundations before committing to a full proposal.

Keywords: recommender systems, transformers, feature interaction, TensorFlow, Keras, deep learning

1 Overview

This document presents a compact technical model describing how a unified recommender-system architecture can merge:

- Transformer-style sequence modeling,
- Feature-interaction modeling (e.g. DeepFM, DCN, DIN),
- A unified TensorFlow/Keras implementation,
- A training pipeline using the Keras fit API.

The goal is to show how modern recommender systems integrate temporal dependencies and structured feature interactions into a single scalable model, leveraging TensorFlow's production-ready ecosystem.

2 1. Minimal Unified Architecture (TensorFlow/Keras)

Below is a minimal Keras model that combines:

- MultiHeadAttention for sequential user behaviour,
- static feature embeddings (user, item, context),
- an MLP for feature interactions.

Model Code

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

class UnifiedRecModel(keras.Model):
    def __init__(
        self,
        num_items,
        num_users,
        num_context_feats,
        d_model=64,
        n_heads=4,
        d_ff=128,
        mlp_hidden=128,
    ):
        super().__init__()

        self.item_emb = layers.Embedding(num_items, d_model)
        self.user_emb = layers.Embedding(num_users, d_model)
        self.ctx_emb = layers.Embedding(num_context_feats, d_model)

        self.attention = layers.MultiHeadAttention(
            num_heads=n_heads,
            key_dim=d_model // n_heads,
        )
        self.ffn = keras.Sequential([
            layers.Dense(d_ff, activation="relu"),
            layers.Dense(d_model),
        ])
        self.norm1 = layers.LayerNormalization()
        self.norm2 = layers.LayerNormalization()

        self.mlp = keras.Sequential([
            layers.Dense(mlp_hidden, activation="relu"),
```

```

        layers.Dense(1),
    ])

def call(self, inputs, training=False):
    seq_items, user_ids, target_items, ctx_feats, seq_mask = inputs

    seq_emb = self.item_emb(seq_items) # (B, T, d)
    user_vec = self.user_emb(user_ids) # (B, d)
    item_vec = self.item_emb(target_items) # (B, d)
    ctx_vec = tf.reduce_mean(
        self.ctx_emb(ctx_feats), axis=1
    ) # (B, d)

    # self-attention with padding mask
    attn_mask = seq_mask[:, tf.newaxis, tf.newaxis, :]
    attn_out = self.attention(
        seq_emb, seq_emb, attention_mask=attn_mask
    )
    seq_enc = self.norm1(seq_emb + attn_out)
    seq_enc = self.norm2(seq_enc + self.ffn(seq_enc))

    # masked mean-pool
    mask_f = tf.cast(seq_mask, tf.float32)
    lengths = tf.reduce_sum(mask_f, axis=1, keepdims=True)
    seq_pool = tf.reduce_sum(
        seq_enc * mask_f[:, :, tf.newaxis], axis=1
    ) / tf.maximum(lengths, 1.0)

    x = tf.concat([seq_pool, user_vec, item_vec, ctx_vec], axis=-1)
    return tf.squeeze(self.mlp(x), axis=-1)

```

3 2. Merging Attention and Feature-Interaction Layers

A unified architecture combines:

- Transformer attention for temporal modelling,
- Feature-interaction layers (e.g. FM, DeepFM) for structured fields.

Attention

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

Factorization Machine Interaction

$$y_{\text{FM}} = \frac{1}{2} \sum_{i < j} (\langle v_i, v_j \rangle).$$

FM Layer Code

```
class FMInteraction(keras.layers.Layer):
    def call(self, field_embs):
        # field_embs: (B, num_fields, d)
        sum_emb = tf.reduce_sum(field_embs, axis=1)
        sum_sq = sum_emb * sum_emb
        sq_sum = tf.reduce_sum(field_embs * field_embs, axis=1)
        fm = 0.5 * tf.reduce_sum(sum_sq - sq_sum, axis=1)
        return fm
```

4 3. Training Pipeline

TensorFlow's `model.compile/model.fit` API provides a concise training pipeline:

1. Data preparation (sequence padding, masks, feature IDs) as `tf.data.Dataset`.
2. Compile with Adam optimiser and binary cross-entropy loss.
3. Call `model.fit` with built-in AUC metric tracking.
4. Export via TF-Serving or TFLite for deployment.

Training Loop

```
model = UnifiedRecModel(
    num_items, num_users, num_context_feats
)

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    loss=tf.keras.losses.BinaryCrossentropy(from_logits=True),
    metrics=[tf.keras.metrics.AUC(name="auc")],
)

model.fit(
    train_dataset,
    validation_data=val_dataset,
    epochs=num_epochs,
```

```
callbacks=[
    tf.keras.callbacks.EarlyStopping(
        monitor="val_auc",
        patience=3,
        restore_best_weights=True,
    )
],
)
```

5 4. TensorFlow Ecosystem and Deployment

- **TF-Serving**: production-grade REST/gRPC model serving with versioning.
- **TFLite**: on-device inference for mobile and edge scenarios.
- **TF Extended (TFX)**: end-to-end ML pipelines with data validation and model monitoring.
- **SavedModel format**: language-agnostic serialisation for cross-platform deployment.
- **Trend**: TensorFlow remains the dominant choice in large-scale industry CTR/CVR systems.